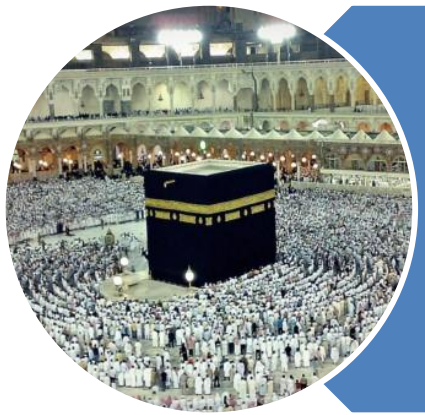
Data Structures - I

CPCS 204

رَبِّ اشْرَحْ لِي صَدْرِي وَيَسِّرْ لِي أَمْرِي وَاحْلُلْ عُقْدَةً مِنْ لِسَانِي يَفْقَهُوا قَوْلِي



O my Lord! Expand for me my chest [with assurance] and ease for me my task and untie the knot from my tongue that they may understand my speech. **(Quran 20 : 25-28)**

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

CPCS 204

Binary Search Tree (BST)

Introduction

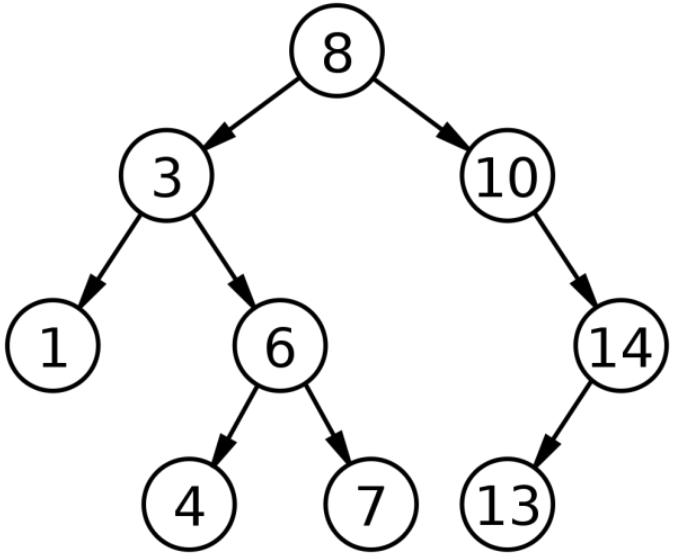
Dr. Jonathan (Yahya) Cazalas

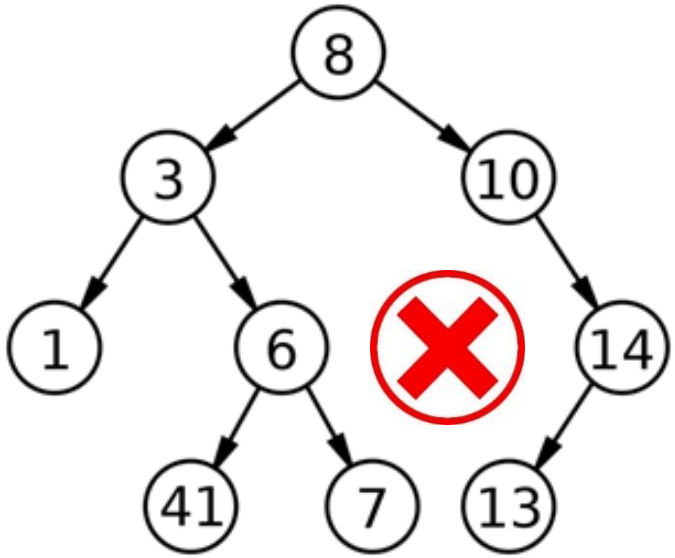
Dr. Muhammad Umair Ramzan

CPCS 204

BST - Introduction

- Value of **left child** is **smaller** than **root**
- Value of **right child** is **greater or equal** to the **root**
- This **order property** is applicable to each and **every node**



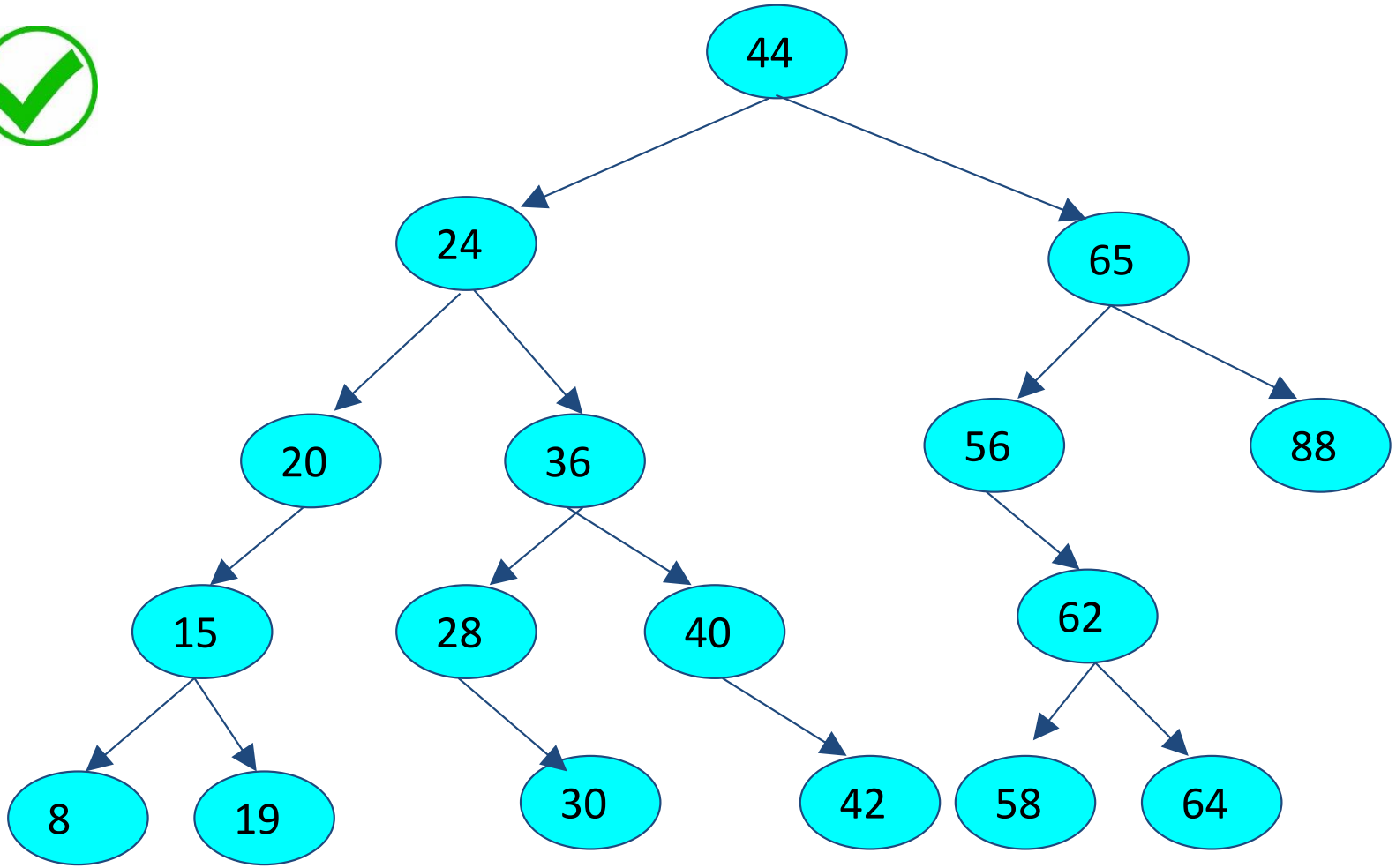


Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

CPCS 204

BST - Introduction



Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

BST - Introduction

- **Summary**
  - Searching is **efficient**
  - **Left** child is always **smaller**
  - **Right** child is always **greater or equal**
  - Searching will take **O ($\log_2$ n)**

CPCS 204

Binary Search Tree (BST)

Operations

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

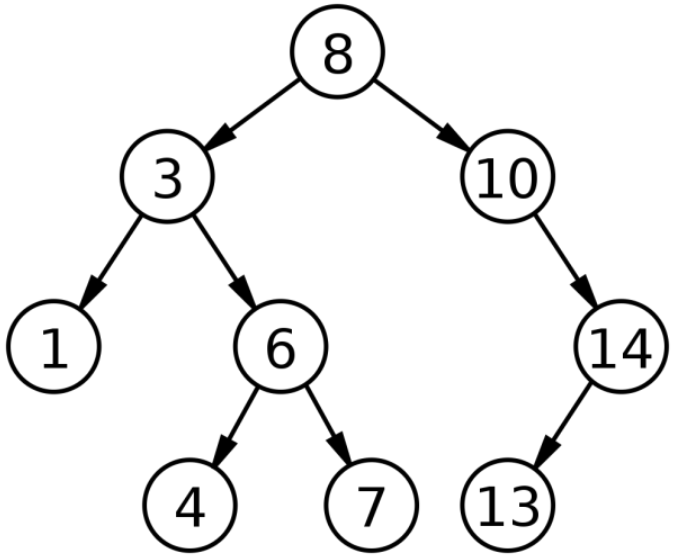
BST - Operations

- **Traversal**
  - Accessing/visiting all values of a tree once
- **Searching**
  - Finding a key value
- **Insertion**
  - Adding a value in a tree
- **Deletion**
  - Removing a value from tree
- **Creation**
  - Creating a tree from empty tree

CPCS 204

BST - Traversal

- **Pre Order Traversal**
  - 8,3,1,6,4,7,10,14,13
- **In Order Traversal**
  - 1,3,4,6,7,8,10,13,14
- **Post Order Traversal**
  - 1,4,7,6,3,13,14,10,8
- **Depth First Search**
  - 8,3,1,6,4,7,10,14,13
- **Breadth First Search**
  - 8,3,10,1,6,14,4,7,13



Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

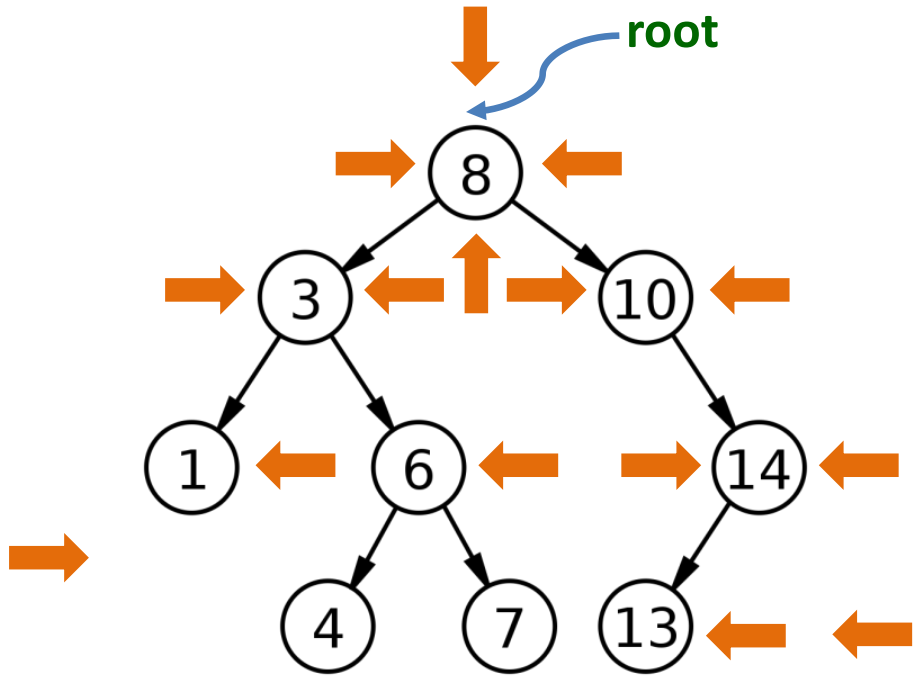
BST - **Searching**

- Order Property should be followed
  1) Start from root
  2) Compare with root
  3) Go right if the key value is greater than or equal to root
  4) Go left if the key value is less than the root
- Keep doing this till you find it
  - return **True**
- or reach to an empty position
  - return **False**

CPCS 204

BST - Searching

- Key = 6
  - True
- Key = 13
  - True
- Key = 0
  - False
- Key = 20
  - False



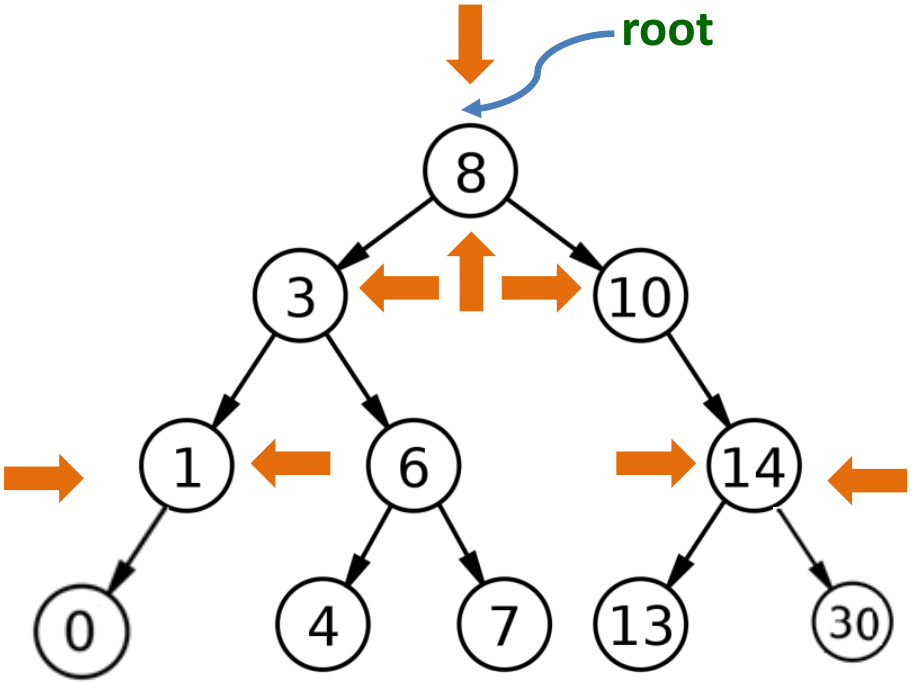
BST - **Insertion**

- Order Property should be maintained
  1) Start from root
  2) Go right if the new value is greater than or equal to the root
  3) Go left if the new value is less than the root
  4) Keep doing this till you come to an empty position
     - **Insert the value**

CPCS 204

BST - Insertion

- Key = 0
  - Null
- Key = 30
  - Null

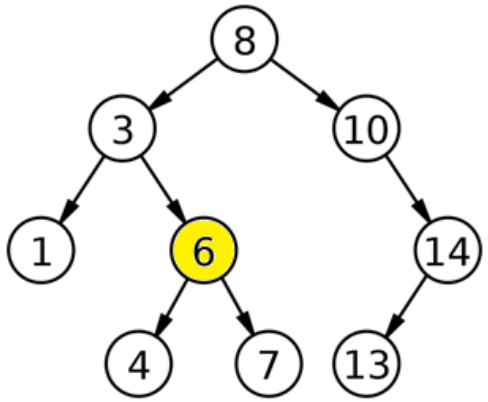


Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

BST - Deletion

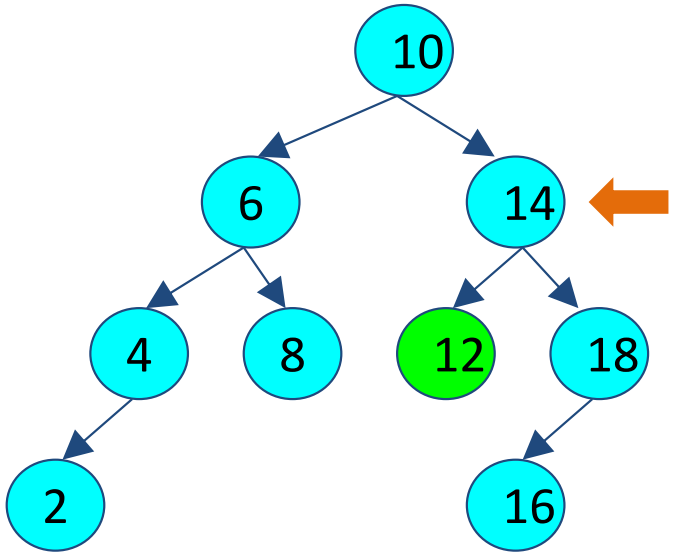
- There are 3 possible cases
  - Deletion of a leaf node
  - Deleting a node with one child
  - Deleting a node with two children

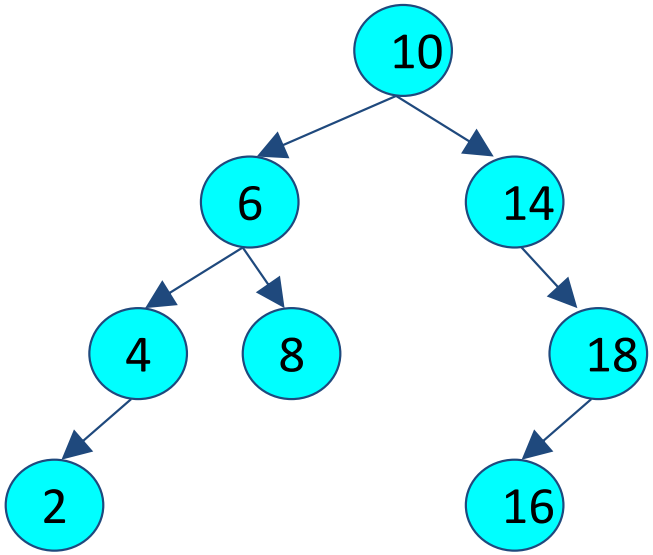


CPCS 204

BST - Deletion – Leaf Node 12

- Identify the parent of the node
  - `parent.left = null;` or
  - or `parent.right = null;`





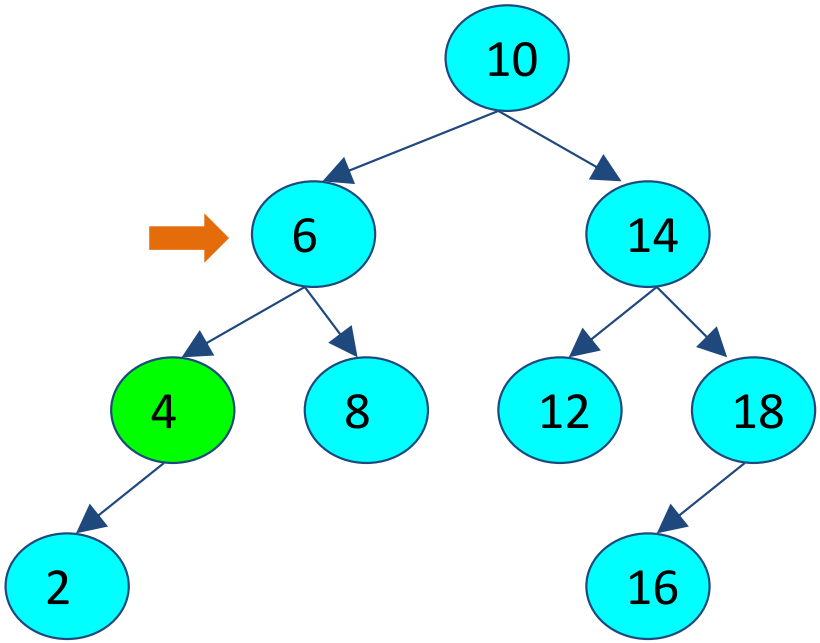
CPCS 204

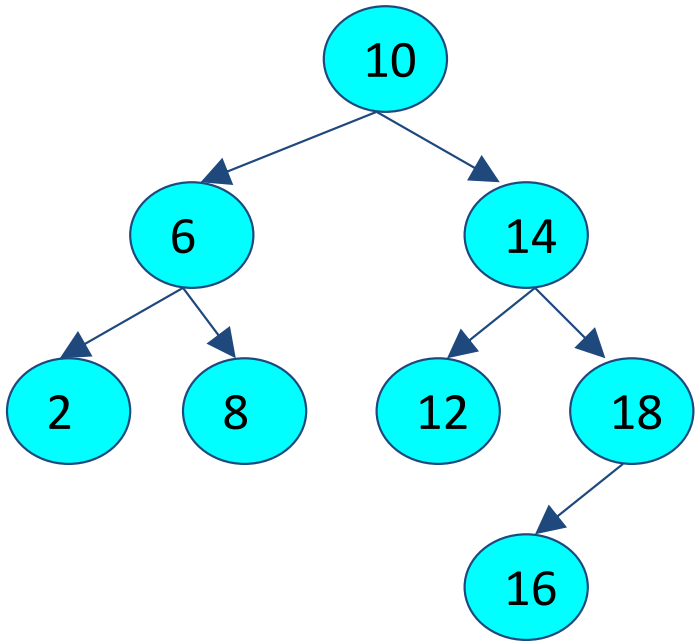
BST - Deletion – Node with One Child

- Identify the parent of the node
- The parent's pointer is changed to the deleted node's child
- This has the effect of lifting up the deleted nodes child by one level in the tree

BST - Deletion – Node with One Child

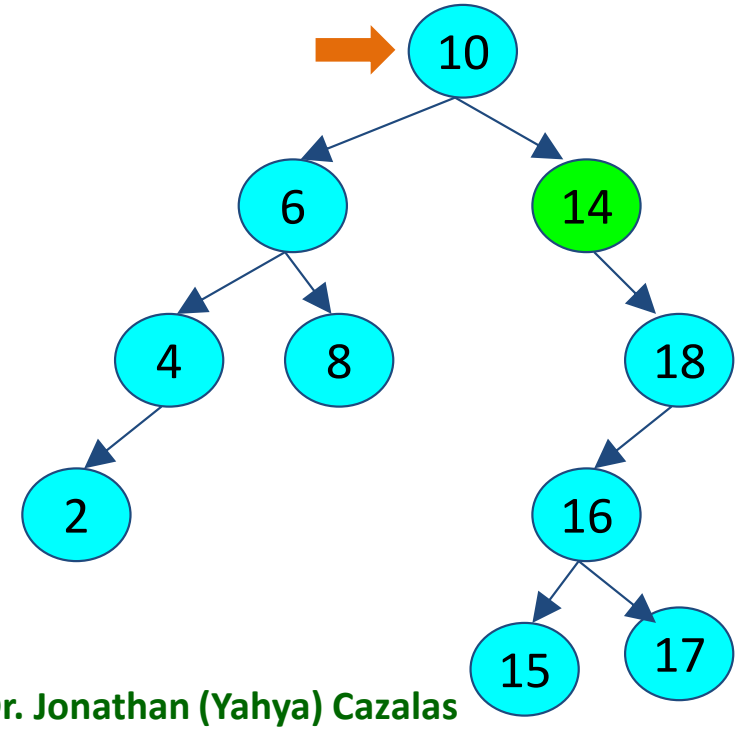
- Identify the parent of the node
  - `parent.left = parent.left.left;`
  - **Or** `parent.left = parent.left.right;`

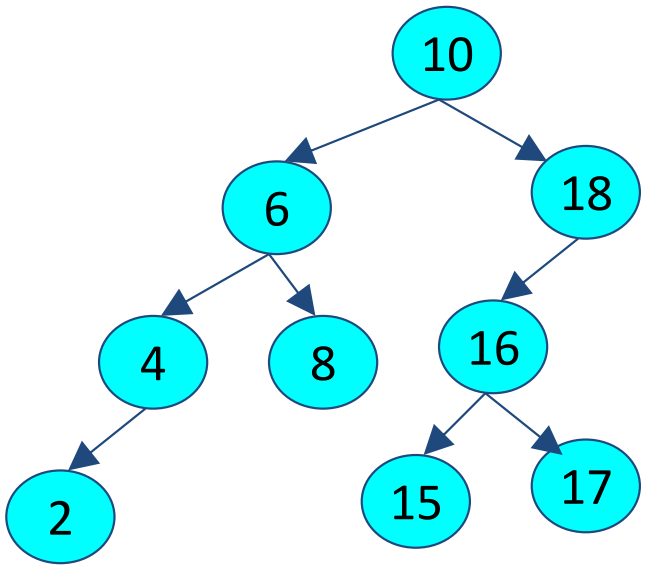




BST - Deletion – Node with One Child

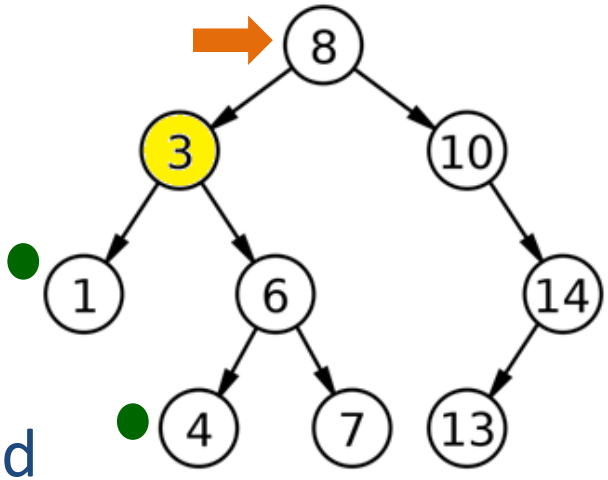
- Identify the parent of the node
  - `parent.right = parent.right.right;`
  - **Or** `parent.right = parent.right.left;`





CPCS 204

BST - Deletion – Node with TWO children



- Identify the parent
- Replace the node to be deleted
  - Largest value of left subtree
    - Right most node of the left child

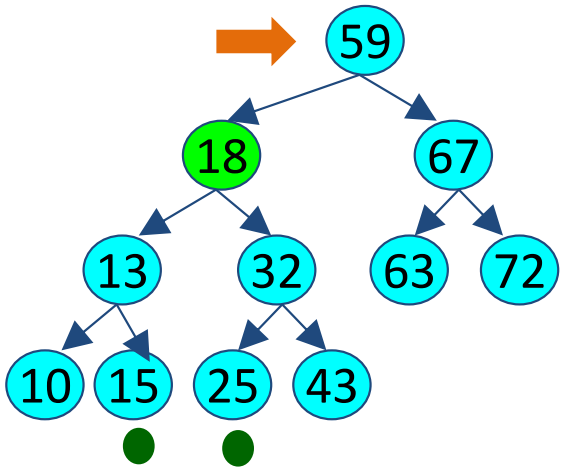
  OR
  - Smallest value of Right subtree
    - Left most node of right child

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

CPCS 204

BST - Deletion – Node with TWO children



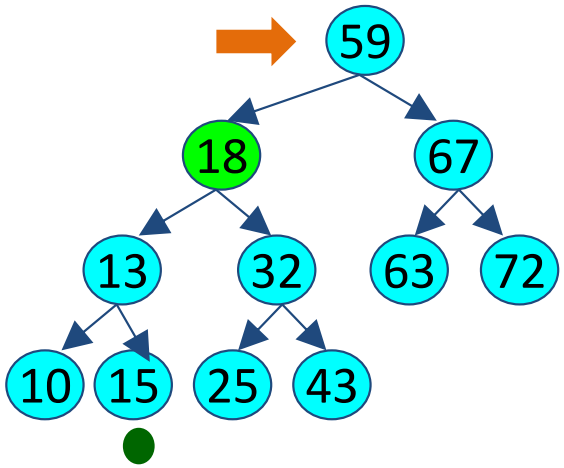
- Identify the parent
- Replace the node to be deleted
  - Largest value of left subtree
    - Right most of the left child

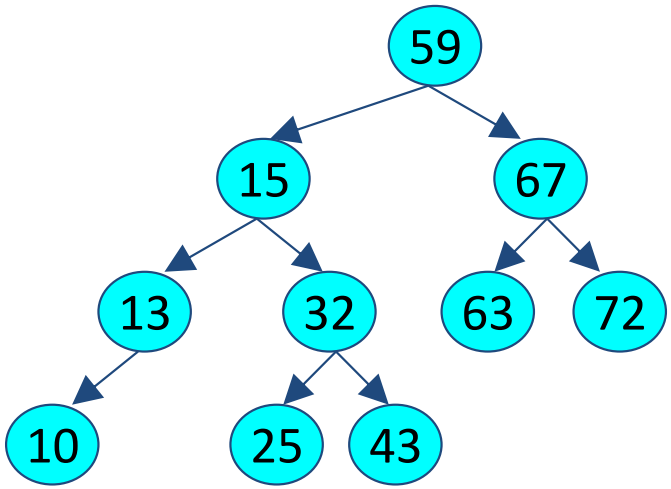
  OR
  - Smallest value of Right subtree
    - Left most of right child

CPCS 204

BST - Deletion – Node with TWO children

- Identify the parent
- Replace the node to be deleted
  - Largest value of left subtree
    - Right most of the left child





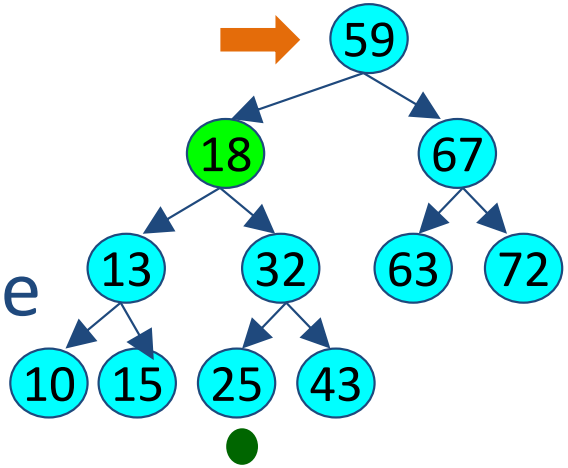
Dr. Jonathan (Yahya) Cazalas

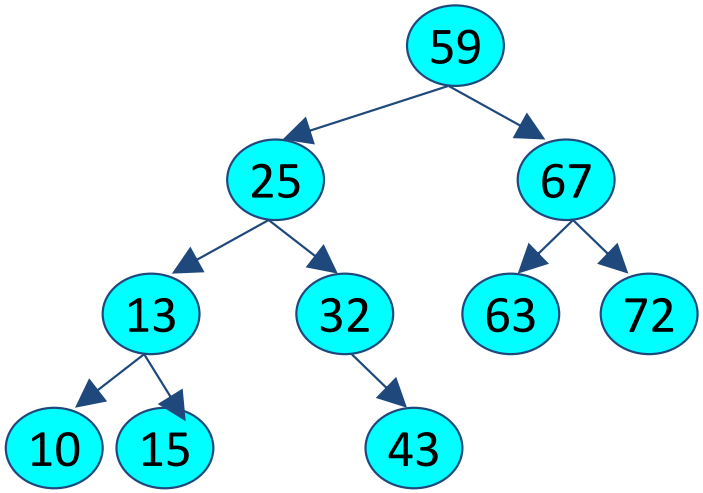
Dr. Muhammad Umair Ramzan

CPCS 204

BST - Deletion – Node with TWO children

- Identify the parent
- Replace the node to be deleted
  - Smallest value of Right subtree
    - Left most of right child



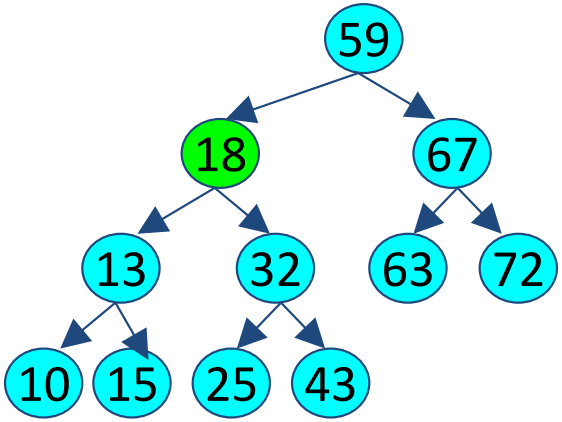


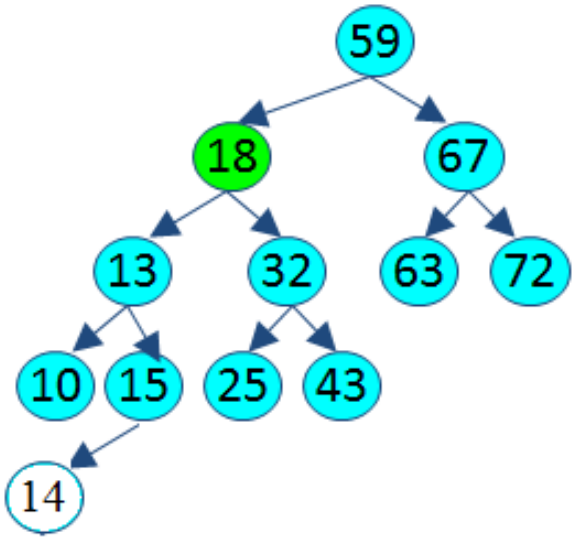
Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

BST - **Deletion** – **Node with TWO children**

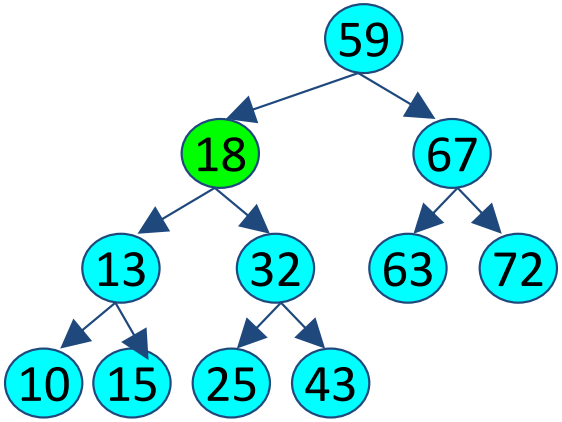
- Largest value of left subtree
  - At most have ONE child
  - Only **left child**

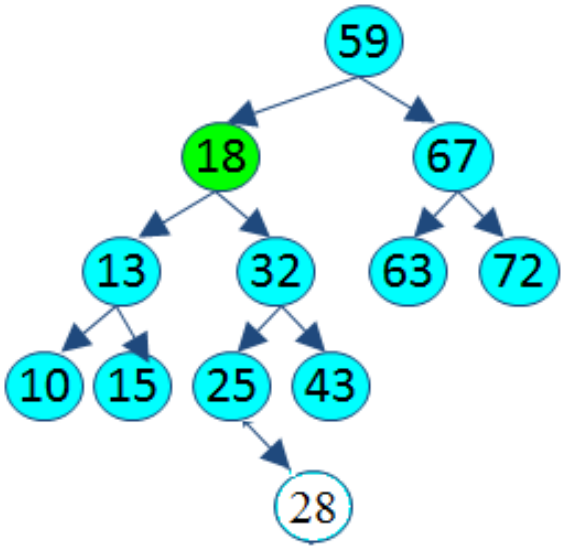




BST - Deletion – Node with TWO children

- Smallest value of left subtree
  - At most have ONE child
  - Only **right child**





Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

BST - **Creation**

- Creating a BST is really nothing more than a series of insertions (calling the insert method over and over)
  - Create a new node
  - Call the insertion method
  - New node will be inserted as a leaf
  - Repeat this process as long as you want to insert

BST - Creation

- Let's assume we insert the following data values, in their order of appearance into an initially empty BST:

  10, 14, 6, 2, 5, 15, and 17

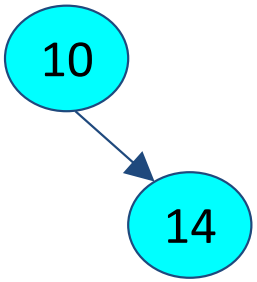
  - **Step 1**:
    - Create a new node with value 10
    - Insert node into tree
    - The tree is currently empty
    - New node becomes the root

CPCS 204

BST - Creation

10, 14, 6, 2, 5, 15, and 17

- **Step 2**:
  - Create a new node with value 14
  - This node belongs in the right subtree of node 10
    - Since 14 > 10
  - The right subtree of node 10 is empty
    - So node 14 becomes the right child of node 10

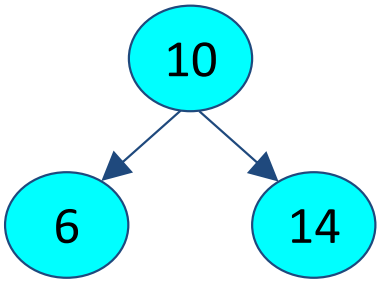


CPCS 204

BST - Creation

10, 14, 6, 2, 5, 15, and 17

- **Step 3**:
  - Create a new node with value 6
  - This node belongs in the left subtree of node 10
    - Since 6 < 10
  - The left subtree of node 10 is empty
    - So node 6 becomes the left child of node 10

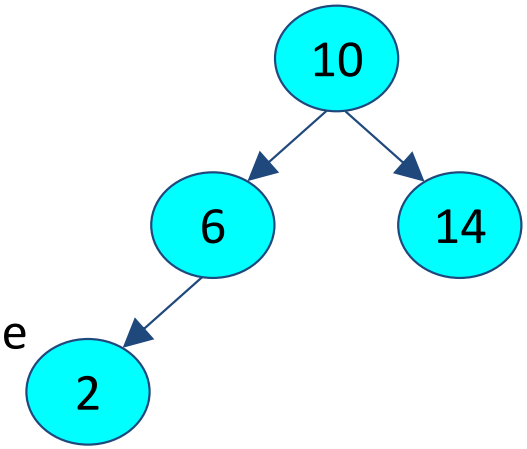


CPCS 204

BST - Creation

10, 14, 6, 2, 5, 15, and 17

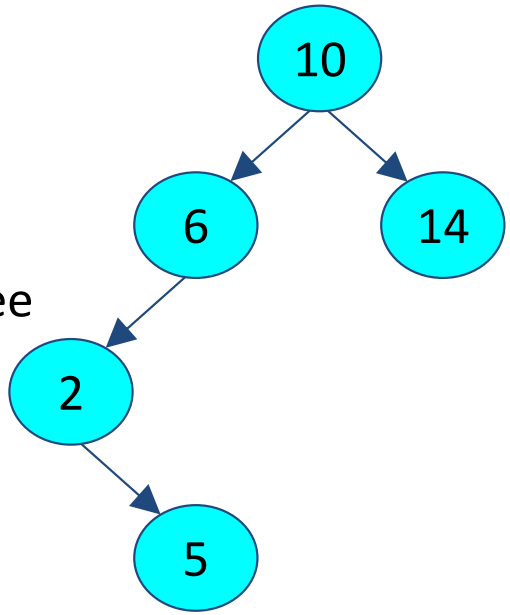
- **Step 4**:
  - Create a new node with value 2
  - This node belongs in the left subtree of node 10
    - Since 2 < 10
  - The root of the left subtree is 6
  - The new node belongs in the left subtree of node 6
    - Since 2 < 6
  - So node 2 becomes the left child of node 6



BST - Creation

10, 14, 6, 2, 5, 15, and 17

- **Step 5**:
  - Create a new node with value 5
  - This node belongs in the left subtree of node 10
    - Since 5 < 10
  - The new node belongs in the left subtree of node 6
    - Since 5 < 6
  - And the new node belongs in the right subtree of node 2
    - Since 5 > 2

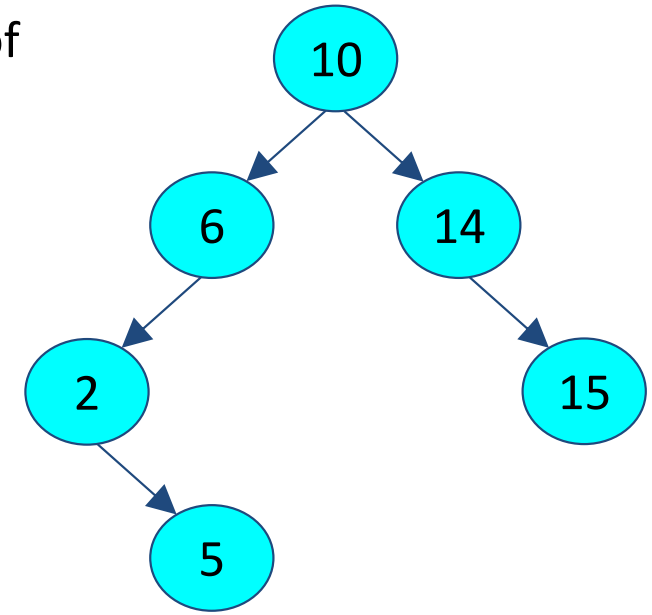


CPCS 204

BST - Creation

10, 14, 6, 2, 5, 15, and 17

- **Step 6**:
  - Create a new node with value 15
  - This node belongs in the right subtree of node 10
    - Since 15 > 10
  - The new node belongs in the right subtree of node 14
    - Since 15 > 14
  - The right subtree of node 14 is empty
  - So node 15 becomes right child of node 14

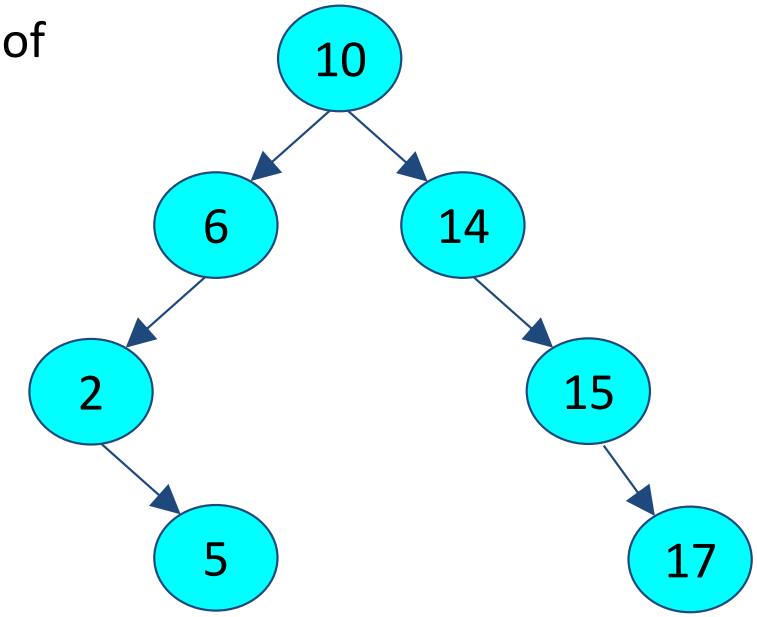


CPCS 204

BST - Creation

10, 14, 6, 2, 5, 15, and 17

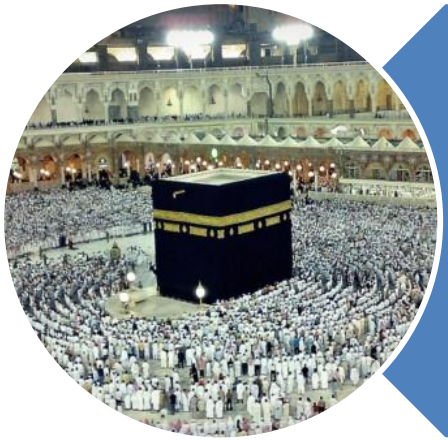
- **Step 7**:
  - Create a new node with value 17
  - This node belongs in the right subtree of node 10
    - Since 17 > 10
  - The new node belongs in the right subtree of node 14
    - Since 17 > 14
  - And the new node belongs in the right subtree of node 15
    - Since 17 > 15



Data Structures - I

"No Bonus" marks for this course anymore

CPCS 204



Allah (Alone) is Sufficient for us, and He is the Best Disposer of affairs (for us).(Quran 3 : 173)

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan